

Design and analysis of algorithm

Week-2 lab assignment

Name – Saksham yadav

Batch-20

Sap id - 590024978

Question 1: Take a random input of 1000 numbers and compare the performance of Linear Search and Binary Search. The program generates random integers, sorts the array for binary search, chooses an existing value as the search key, and measures the time required by both search techniques.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define SIZE 1000
#define REPEAT 1000000

// Comparison function for qsort()
int compare(const void *a, const void *b)
{
    int x = *(const int *)a;
    int y = *(const int *)b;

    if (x < y)
        return -1;
    else if (x > y)
        return 1;
    else
        return 0;
}

// Linear Search
int linearSearch(int arr[], int size, int key)
{
    for (int i = 0; i < size; i++)
    {
        if (arr[i] == key)
        {
            return i;
        }
    }

    return -1;
}

// Binary Search
```

```

int binarySearch(int arr[], int size, int key)
{
    int low = 0;
    int high = size - 1;

    while (low <= high)
    {
        int mid = low + (high - low) / 2;

        if (arr[mid] == key)
        {
            return mid;
        }
        else if (arr[mid] < key)
        {
            low = mid + 1;
        }
        else
        {
            high = mid - 1;
        }
    }

    return -1;
}

int main()
{
    int arr[SIZE];
    int key;
    int result;

    clock_t start, end;

    double linearTime;
    double binaryTime;

    // Initialize random number generator
    srand((unsigned int)time(NULL));

    // Generate 1000 random numbers
    for (int i = 0; i < SIZE; i++)
    {
        arr[i] = rand() % 10000;
    }

    // Select an existing random element as the search key
    key = arr[rand() % SIZE];

```

```

printf("Search Key: %d\n\n", key);

// -----
// Linear Search
// -----

start = clock();

for (int i = 0; i < REPEAT; i++)
{
    result = linearSearch(arr, SIZE, key);
}

end = clock();

linearTime = (double)(end - start) / CLOCKS_PER_SEC;

if (result != -1)
{
    printf("Linear Search: Key found\n");
    printf("Time taken: %.6f seconds\n\n", linearTime);
}

// Sort the array for Binary Search
qsort(arr, SIZE, sizeof(int), compare);

// -----
// Binary Search
// -----

start = clock();

for (int i = 0; i < REPEAT; i++)
{
    result = binarySearch(arr, SIZE, key);
}

end = clock();

binaryTime = (double)(end - start) / CLOCKS_PER_SEC;

if (result != -1)
{
    printf("Binary Search: Key found\n");
    printf("Time taken: %.6f seconds\n\n", binaryTime);
}

```

```
// Compare performance
if (binaryTime < linearTime)
{
    printf("Binary Search is faster.\n");
}
else if (linearTime < binaryTime)
{
    printf("Linear Search is faster.\n");
}
else
{
    printf("Both searches took approximately the same time.\n");
}

return 0;
}
```

Explanation:

The program generates 1000 random numbers and selects one existing value as the search key. Linear Search checks elements one by one, while Binary Search first requires the array to be sorted and then repeatedly divides the search range into half.

Result:

Binary Search is generally faster than Linear Search for a sorted array.

Why Binary Search is Faster:

Linear Search may check up to all 1000 elements, so its time complexity is **O(n)**. Binary Search cuts the search range in half after every comparison, so its time complexity is **O(log n)**. For 1000 elements, Binary Search needs only about **10 comparisons** in the worst case.

Conclusion:

Binary Search is more efficient than Linear Search when the data is sorted.

Question 2: Write a C program to generate 1000 random numbers and compare the execution time of Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort. Display the time taken by each sorting algorithm and plot a comparison graph using Python. The same original random array is copied before each test so that every sorting method receives identical input data.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define SIZE 10000

// Copy original array
void copyArray(int source[], int destination[])
{
    for (int i = 0; i < SIZE; i++)
    {
        destination[i] = source[i];
    }
}

// Bubble Sort
void bubbleSort(int arr[])
{
    for (int i = 0; i < SIZE - 1; i++)
    {
        for (int j = 0; j < SIZE - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

// Selection Sort
void selectionSort(int arr[])
```

```

{
    for (int i = 0; i < SIZE - 1; i++)
    {
        int minIndex = i;

        for (int j = i + 1; j < SIZE; j++)
        {
            if (arr[j] < arr[minIndex])
            {
                minIndex = j;
            }
        }

        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }
}

// Insertion Sort
void insertionSort(int arr[])
{
    for (int i = 1; i < SIZE; i++)
    {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;
    }
}

// Merge function
void merge(int arr[], int left, int mid, int right)
{
    int size = right - left + 1;
    int *temp = (int *)malloc(size * sizeof(int));

    int i = left;
    int j = mid + 1;
    int k = 0;

    while (i <= mid && j <= right)

```

```

{
    if (arr[i] <= arr[j])
    {
        temp[k++] = arr[i++];
    }
    else
    {
        temp[k++] = arr[j++];
    }
}

while (i <= mid)
{
    temp[k++] = arr[i++];
}

while (j <= right)
{
    temp[k++] = arr[j++];
}

for (i = 0; i < size; i++)
{
    arr[left + i] = temp[i];
}

free(temp);
}

// Merge Sort
void mergeSort(int arr[], int left, int right)
{
    if (left < right)
    {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

// Partition for Quick Sort
int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;

```

```

    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;

            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

// Quick Sort
void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pivotIndex = partition(arr, low, high);

        quickSort(arr, low, pivotIndex - 1);
        quickSort(arr, pivotIndex + 1, high);
    }
}

int main()
{
    int original[SIZE];
    int arr[SIZE];

    clock_t start, end;

    double bubbleTime;
    double selectionTime;
    double insertionTime;
    double mergeTime;
    double quickTime;

    srand((unsigned int)time(NULL));

    // Generate 10,000 random numbers

```

```
for (int i = 0; i < SIZE; i++)
{
    original[i] = rand() % 100000;
}

// Bubble Sort
copyArray(original, arr);

start = clock();
bubbleSort(arr);
end = clock();

bubbleTime = (double)(end - start) / CLOCKS_PER_SEC;

// Selection Sort
copyArray(original, arr);

start = clock();
selectionSort(arr);
end = clock();

selectionTime = (double)(end - start) / CLOCKS_PER_SEC;

// Insertion Sort
copyArray(original, arr);

start = clock();
insertionSort(arr);
end = clock();

insertionTime = (double)(end - start) / CLOCKS_PER_SEC;

// Merge Sort
copyArray(original, arr);

start = clock();
mergeSort(arr, 0, SIZE - 1);
end = clock();

mergeTime = (double)(end - start) / CLOCKS_PER_SEC;

// Quick Sort
copyArray(original, arr);

start = clock();
quickSort(arr, 0, SIZE - 1);
end = clock();
```

```

quickTime = (double)(end - start) / CLOCKS_PER_SEC;

// Display execution times
printf("\nSorting Algorithm Performance\n");
printf("-----\n");

printf("Bubble Sort      : %.6f seconds\n", bubbleTime);
printf("Selection Sort : %.6f seconds\n", selectionTime);
printf("Insertion Sort : %.6f seconds\n", insertionTime);
printf("Merge Sort      : %.6f seconds\n", mergeTime);
printf("Quick Sort      : %.6f seconds\n", quickTime);

// Save results to CSV file
FILE *file = fopen("sorting_times.csv", "w");

if (file != NULL)
{
    fprintf(file, "Algorithm,Time\n");
    fprintf(file, "Bubble Sort,%.6f\n", bubbleTime);
    fprintf(file, "Selection Sort,%.6f\n", selectionTime);
    fprintf(file, "Insertion Sort,%.6f\n", insertionTime);
    fprintf(file, "Merge Sort,%.6f\n", mergeTime);
    fprintf(file, "Quick Sort,%.6f\n", quickTime);

    fclose(file);

    printf("\nResults saved to sorting_times.csv\n");
}
else
{
    printf("\nError creating CSV file.\n");
}

return 0;
}

```

Code for graph:

```

import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path

# Get CSV file from the same folder as this Python file
file_path = Path(__file__).with_name("sorting_times.csv")

# Read CSV
data = pd.read_csv(file_path)

```

```
print(data)

# Plot graph
plt.bar(data["Algorithm"], data["Time"])

plt.xlabel("Sorting Algorithm")
plt.ylabel("Execution Time (seconds)")
plt.title("Comparison of Sorting Algorithms")

plt.xticks(rotation=20)
plt.tight_layout()

plt.show()
```

Explanation:

The program generates 10,000 random numbers and compares the execution time of Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort. Before testing each algorithm, the same original array is copied so that every sorting method receives identical input.

Result:

Bubble Sort, Selection Sort, and Insertion Sort take more time because their average time complexity is $O(n^2)$. Merge Sort and Quick Sort are much faster because their average time complexity is $O(n \log n)$.

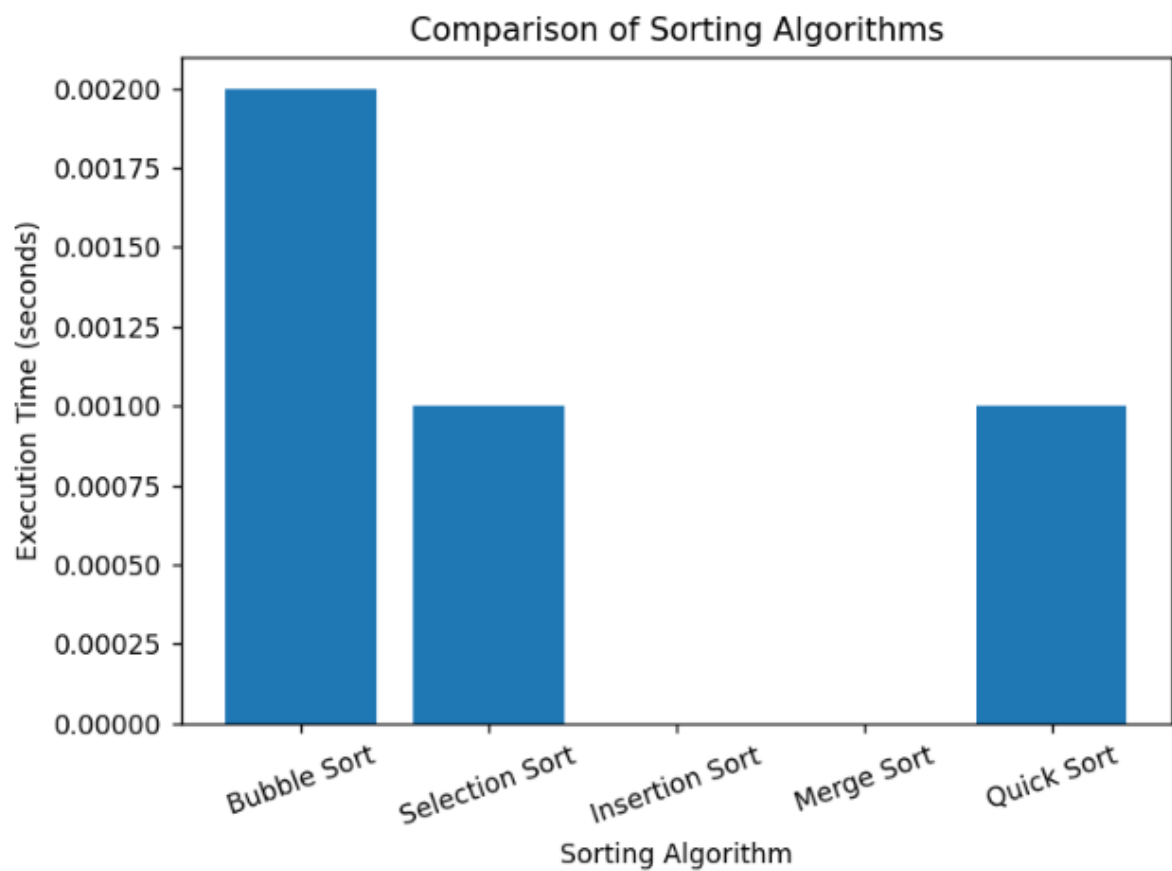
Why Merge Sort and Quick Sort are Faster:

They divide the problem into smaller parts instead of repeatedly comparing many elements one by one. Because of this, they perform much better when the number of elements becomes large.

Conclusion:

For large datasets, Merge Sort and Quick Sort are generally more efficient than Bubble Sort, Selection Sort, and Insertion Sort. The graph also clearly shows the difference in execution time between the sorting algorithms.

Figure 1



(x, y) = (Selection Sort, 0.001793)